

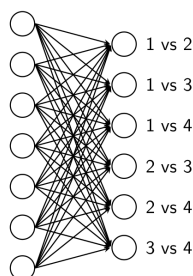
LAB 4: Softmax Regression

Giới thiệu

Trong thực tế, các bài toán phân loại thường có nhiều hơn 2 nhãn dữ liệu, gọi chung là *phân loại đa lớp* (multiclass classification). Ví dụ, trong bài toán phân loại hình ảnh chữ số viết tay, ta sẽ cần phân loại các hình ảnh chữ số từ 0 đến 9, tức là có 10 nhãn khác nhau. Ta có thể sử dụng hồi quy logistic cho bài toán này, nhưng hồi quy logistic chỉ có thể phân loại 2 nhãn. Để giải quyết vấn đề này, một cách tự nhiên, ta sẽ có các hướng tiếp cận thông qua một số kỹ thuật:

1. *One-versus-one*

Áp dụng hồi quy logistic (binary classification) cho từng cặp nhãn. Ví dụ, với 4 nhãn A, B, C ta sẽ sử dụng hồi quy logistic cho các cặp (A, B), (A, C), (B, C). Tổng quát, với K nhãn, ta sẽ có $\frac{K(K-1)}{2}$ cặp (bộ phân loại). Sau đó, với mỗi điểm dữ liệu cần kiểm tra, ta

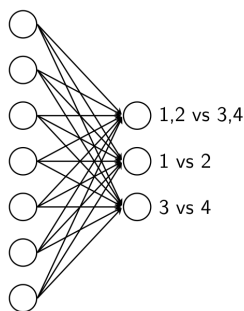


Hình 1: Hồi quy logistic cho từng cặp nhãn

sẽ dùng tất cả bộ phân loại để dự đoán nhãn của nó. Kết quả cuối cùng sẽ là nhãn có số phiếu bầu cao nhất. Có thể thấy rằng, với cách làm này, ta cần có rất nhiều bộ phân loại cần phải xây dựng và lưu trữ, gây tốn kém về bộ nhớ và thời gian tính toán.

2. *Hierarchical*

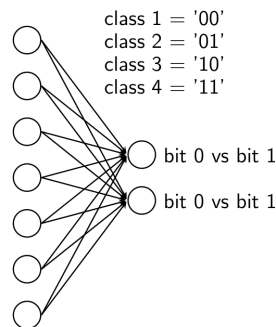
Để giảm số lượng bộ phân loại cần xây dựng trong phương pháp trên, ta có thể xây dựng một cây nhị phân, trong đó mỗi nhãn sẽ là một nút lá. Mỗi nhãn sẽ được phân loại theo thứ tự từ gốc đến lá. Ví dụ, với 4 nhãn A, B, C, D ta có thể xây dựng cây như sau:



Hình 2: Hierarchical

3. Binary coding

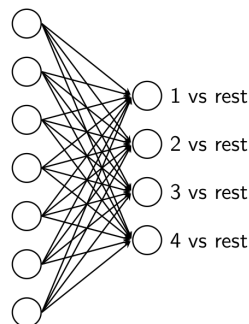
Một cách tiếp cận khác để giảm số lượng bộ phân loại là sử dụng mã nhị phân cho các nhãn. Ví dụ, nếu có 4 classes thì các nhãn sẽ được mã hóa lần lượt là 00, 01, 10, 11.



Hình 3: Mã hóa nhị phân cho 4 nhãn

4. One-versus-rest

One-vs-rest (còn được gọi là one-hot coding, one-vs-all, one-against-rest,...) là phương pháp phổ biến nhất được sử dụng trong bài toán phân loại đa lớp. Cụ thể, nếu ta mã hóa các lớp đầu ra bằng K bit, lớp thứ i sẽ ứng với bit thứ i bằng 1 và tất cả các bit còn lại bằng 0. Ví dụ: nếu có 4 lớp 1, 2, 3, 4 thì các lớp sẽ được mã hóa lần lượt là [1000], [0100], [0010], [0001]. Như vậy, ta sẽ cần xây dựng K bộ phân loại nhị phân để phân loại K lớp. Trong đó, mỗi bộ phân loại sẽ được huấn luyện để phân loại lớp thứ i với tất cả các lớp còn lại.



Hình 4: one-versus-rest

I. Softmax Regression

Bài 1: Xây dựng class SoftmaxRegression

1.1. Cài đặt hàm softmax

Hàm softmax được định nghĩa như sau:

$$\sigma(z) = \frac{e^z}{\sum_{k=1}^K e^{z_k}} \quad (1)$$

Trong đó:

- z là đầu vào có kích thước $(K,)$.
- $\sigma(z)$ là đầu ra của hàm softmax.
- K là số lớp (classes).

Hàm softmax sẽ chuyển đổi đầu vào z thành xác suất cho mỗi lớp. Đầu ra của hàm softmax sẽ có kích thước $(K,)$ và tổng xác suất của các lớp trong mỗi đầu ra sẽ bằng 1.

```
1 def softmax(z):
2     exp_z = np.exp(z)
3     sum_exp_z = np.sum(exp_z, axis=1, keepdims=True)
4     return exp_z / sum_exp_z
```

1.2. Cài đặt loss function

Hàm mất mát cho hồi quy softmax (cross-entropy) được định nghĩa như sau:

$$L(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log(\hat{y}_{ik}) \quad (2)$$

Trong đó:

- N là số lượng mẫu trong tập huấn luyện.
- K là số lớp (classes).
- y_{ik} là nhãn thực tế của mẫu thứ i cho lớp thứ k .
- $\hat{y}_{ik}$ là xác suất dự đoán của mẫu thứ i cho lớp thứ k .

```
1 def cross_entropy(y, y_pred):
2     loss = -np.sum(y * np.log(y_pred))/y.shape[0]
3     return loss
```

1.3. Hoàn thiện class *SoftmaxRegression*

```

1 class SoftmaxRegression:
2     def __init__(self, learning_rate=0.01, epochs=1000):
3         self.lr = learning_rate
4         self.epochs = epochs
5         self.W = None
6         self.b = None
7
8     def softmax(self, z):
9         exp_z = np.exp(z)
10        sum_exp_z = np.sum(exp_z, axis=1, keepdims=True)
11        return exp_z / sum_exp_z
12
13    def cross_entropy(self, y, y_pred):
14        loss = -np.sum(y * np.log(y_pred)) / y.shape[0]
15        return loss
16
17    def predict(self, X):
18        z = np.dot(X, self.W) + self.b
19        y_pred = softmax(z)
20        return np.argmax(y_pred, axis=1)
21
22    def fit(self, X, y):
23        pass

```

Bài 2: Passive Mine classification

Sử dụng class *SoftmaxRegression* đã xây dựng ở trên để huấn luyện mô hình phân loại trên dataset Land Mines dataset được thu thập bởi nhóm nghiên cứu tại Đại học Gazi, Thổ Nhĩ Kỳ.

Bài 3: Huấn luyện mô hình phân loại chữ số viết tay MNIST

MNIST là một dataset nổi tiếng trong lĩnh vực học máy, được thu thập và công bố bởi Yann LeCun cùng cộng sự vào năm 1998. MNIST bao gồm 70.000 hình ảnh chữ số viết tay từ 0 đến 9, với kích thước mỗi hình ảnh là 28x28 pixel.

3.1. Tải tập dữ liệu MNIST

```

1 from tensorflow.keras.datasets import mnist

```

3.2. Tiền xử lý dữ liệu

3.2.1. Chia tập dữ liệu thành tập huấn luyện và tập kiểm tra

```

1 (X_train, y_train), (X_test, y_test) = mnist.load_data()
2 print(X_train.shape, y_train.shape)
3 print(X_test.shape, y_test.shape)

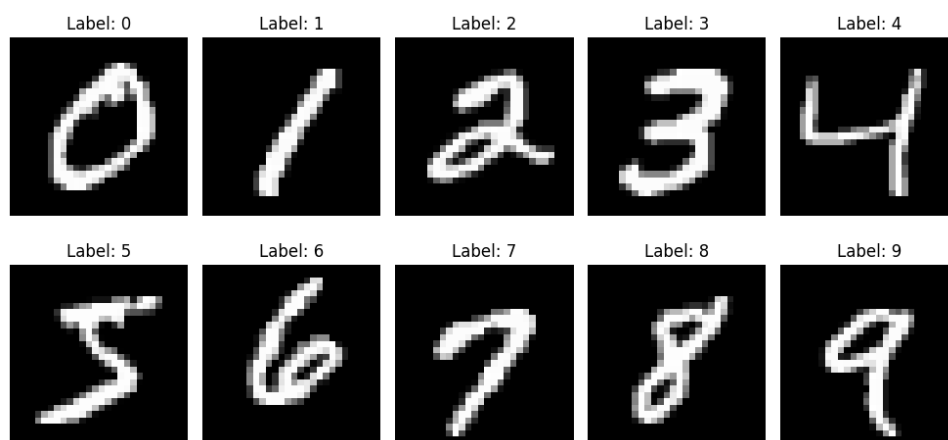
```

3.2.2. In ra 10 hình ảnh đầu tiên của mỗi nhãn

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 unique_labels = np.unique(y_train)
5 print("Unique labels:", unique_labels)
6
7 plt.figure(figsize=(10, 5))
8 for label in unique_labels:
9     index = np.where(y_train == label)[0][0]
10    plt.subplot(2, 5, label + 1)
11    plt.imshow(X_train[index], cmap='gray')
12    plt.title(f'Label: {label}')
13    plt.axis('off')
14 plt.tight_layout()
15 plt.show()

```



3.2.3. Chuẩn hóa dữ liệu

Do giá trị của các pixel trong ảnh giao động trong khoảng $[0, 255]$, các giá trị này sẽ ảnh hưởng đến quá trình huấn luyện mô hình. Để khắc phục vấn đề này, ta cần chuẩn hóa dữ liệu về khoảng $[0, 1]$ để giúp cho quá trình huấn luyện mô hình diễn ra nhanh và hiệu quả hơn.

```

1 X_train = X_train.reshape(X_train.shape[0], -1) / 255.0
2 X_test = X_test.reshape(X_test.shape[0], -1) / 255.0
3 print("X_train shape:", X_train.shape)
4 print("X_test shape:", X_test.shape)

```

3.2.4. Chuyển đổi nhãn thành one-hot encoding

Đối với bài toán phân loại đa lớp, ta cần chuyển đổi nhãn thành dạng one-hot encoding để có thể sử dụng hàm mất mát cross-entropy. Ví dụ, với 3 nhãn 0, 1, 2, ta sẽ chuyển đổi thành:

- Nhãn 0: $[1, 0, 0]$
- Nhãn 1: $[0, 1, 0]$

- Nhân 2: $[0, 0, 1]$

Hãy thực hiện cài đặt hàm one-hot encoding:

```
1 def one_hot_encoding(y, num_classes):
2     pass
```

3.3. Huấn luyện và đánh giá mô hình

3.3.1. Huấn luyện mô hình

Hãy huấn luyện mô hình với class SoftmaxRegression đã xây dựng ở trên. Thực hiện thay đổi các tham số như learning rate, epochs,... để có được độ chính xác tốt nhất trên tập kiểm tra.

3.3.2. Đánh giá mô hình

Sử dụng tập kiểm tra để đánh giá mô hình thông qua các chỉ số đã học trong bài Lab 7 (Accuracy, Precision, Recall, F1-score) và vẽ confusion matrix, sau đó đưa ra nhận xét.

3.4. Dự đoán chữ số viết tay trên dữ liệu thực tế

- Thực hiện chụp một hình ảnh chữ số viết tay bất kỳ (Lưu ý tới sự tương đồng về dữ liệu huấn luyện và dữ liệu thực tế).
- Tiền xử lý hình ảnh tương tự như các bước đã thực hiện với tập dữ liệu MNIST.
- Sử dụng mô hình đã huấn luyện để dự đoán chữ số viết tay trong hình ảnh.
- Hiển thị hình ảnh và kết quả dự đoán.
- Nhận xét về độ chính xác của mô hình.

Bài 4: Huấn luyện mô hình phân loại trang phục Fashion MNIST

Được ra đời vào năm 2017 bởi Zalando Research, Fashion MNIST là một tập dữ liệu hình ảnh thuộc chủ đề thời trang, được xây dựng để thay thế cho tập dữ liệu MNIST. Tương tự với MNIST, tập dữ liệu này cũng bao gồm 70.000 hình ảnh grayscale, nhưng thay vì là các chữ số viết tay, các hình ảnh trong tập dữ liệu thuộc nhóm 10 trang phục thời trang quen thuộc như áo thun, quần jeans, giày thể thao, v.v.

Tải tập dữ liệu Fashion MNIST

```
1 from tensorflow.keras.datasets import fashion_mnist
2 (X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()
3 print(X_train.shape, y_train.shape)
4 print(X_test.shape, y_test.shape)
```



Thực hiện các yêu cầu sau:

- Thực hành tiền xử lý và huấn luyện tương tự như bài 2, nhưng trên tập dữ liệu Fashion MNIST.
- Đưa ra kết quả của các chỉ số đánh giá mô hình trên tập kiểm tra, vẽ confusion matrix và nhận xét.
- Thực hiện chụp một hình ảnh trang phục bất kỳ (Lưu ý tới sự tương đồng về dữ liệu huấn luyện và dữ liệu thực tế).
- Tiền xử lý hình ảnh tương tự như các bước đã thực hiện với tập dữ liệu Fashion MNIST.
- Sử dụng mô hình đã huấn luyện để dự đoán trang phục trong hình ảnh.
- Hiển thị hình ảnh và kết quả dự đoán.
- Nhận xét về độ chính xác của mô hình.